Exp. 2: Power Simulation (MPT Model with Response Time Extension)

Annemarie Oelert

May 26, 2026

Specification of the model:
Each equation is coded as `Condition.Response`, with the response being composed of positive (p) vs. negative (n) binary postratings and fast (f) vs. slow (s) responses.

Parameters are t (attribution), s (association), g (guessing positive) and f_t / f_s / f_g (responding fast within the respective process).

```
library(mpt)

specRT <- mptspec(
    OwnWin.pf = t*f_t + (1-t)*s*f_s + (1-t)*(1-s)*g*f_g,
    OwnWin.ps = t*(1-f_t) + (1-t)*s*(1-f_s) + (1-t)*(1-s)*g*(1-f_g),
    OwnWin.nf = (1-t)*(1-s)*(1-g)*f_g,
    OwnWin.ns = (1-t)*(1-s)*(1-g)*(1-f_g),
    OwnLoss.pf = (1-t)*(1-s)*g*f_g,
    OwnLoss.ps = (1-t)*(1-s)*g*(1-f_g),
    OwnLoss.nf = t*f_t + (1-t)*s*f_s + (1-t)*(1-s)*(1-g)*f_g,
    OwnLoss.ns = t*(1-f_t) + (1-t)*s*(1-f_s) + (1-t)*(1-s)*(1-g)*(1-f_g),
    OppWin.pf = (1-t)*s*f_s + (1-t)*(1-s)*g*f_g,
    OppWin.ps = (1-t)*s*(1-f_s) + (1-t)*(1-s)*g*(1-f_g),
    OppWin.nf = t*f_t + (1-t)*(1-s)*(1-g)*f_g,
    OppWin.ns = t*(1-f_t) + (1-t)*(1-s)*(1-g)*(1-f_g),
    OppLoss.pf = t*f_t + (1-t)*(1-s)*g*f_g,
    OppLoss.ps = t*(1-f_t) + (1-t)*(1-s)*g*(1-f_g),
    OppLoss.nf = (1-t)*s*f_s + (1-t)*(1-s)*(1-g)*f_g,
    OppLoss.ns = (1-t)*s*(1-f_s) + (1-t)*(1-s)*(1-g)*(1-f_g)
)
```

Each participant contributes two data points per condition. With four conditions, that means 8 CSs per participant and a total of 140 data points per condition.

```
nSubj <- 70
nTrials <- 2
n <- nSubj*nTrials

m <- structure(list(
        treeid = specRT$treeid,
        n = setNames(rep(n, 16), specRT$treeid),
        pcat = specRT$par2prob(  # has to be in THAT order!
                    c(t=0.15, f_t=0.3, s=0.15, f_s=0.7, g=0.5, f_g=0.5))
), class = "mpt")
```

The power is simulated for the test of H0: `s=0`.

```
pval <- replicate(5000, {
    y <- simulate(m)
    m1 <- mpt(specRT, data=y, method="EM", optimargs=list(maxit=2000))
    m2 <- mpt(update(specRT, .restr=list(s=0, f_s=0.5)), data=y, method="EM",
              optimargs=list(maxit=2000))
    anova(m2, m1)[["Pr(>Chi)"]][2]
})
mean(pval < .05)
```

```
## [1] 0.8482
```